PITCH X Growth Engine

Developer Documentation & Execution Guide

trypitch.co / @trypitchdotco

August 2026

Webhooks · Dispatched Skills · Business Accomplishments (ROI) · Schedules & Cron · Setup

1 Introduction & System Overview

The PITCH X Growth Engine runs the X/Twitter growth-to-sales funnel for https://trypitch.co — an AI video editor that turns task descriptions into studio-quality narrated demo MP4s. The engine watches mentions of `@trypitchdotco`, turns them into real product demo videos, delivers them back in-thread, and runs a full outbound growth funnel (prospecting, warm-up, outreach, content, community) on top of a real human personality.

Everything runs from one compiled Rust binary (`pitch-cli`), a local SQLite database, and an **opencode session dispatcher**:

- **`pitch-cli server`** — an Axum webhook dispatcher listening on `0.0.0.0:8790` (`PORT`). Every incoming event **spawns a fresh `opencode run` session** (capped by `MAX_OPENCODE_SESSIONS`, default 3) that executes the matching skill. There is no in-Rust mention pipeline anymore: the old `inbox`, `worker`, and `pitch_mcp` modules were deleted.
- **Skills** (`.opencode/skills/x-*`) — each flow is a skill the dispatched session loads: `x-growth` (orchestrator), `x-prospect`, `x-engage`, `x-outreach`, `x-content`, `x-community`, `x-mention`.
- **MCP servers** — `pitch` (remote `https://api.trypitch.co/mcp`, `create_demo_video` / `get_job` / `get_credits`) and `xmcp` (official X MCP via the local `xurl` bridge to `https://api.x.com/mcp`) for X reads.
- **agent-webbridge** — drives the user's **real Chrome** (`Testing` profile, router `127.0.0.1:10086`). **All X writes go through the browser**, never a headless X API client.
- **SQLite** (`data/pitch_bot.db`, WAL mode) — CRM + mention-job pipeline memory.

Two ways work gets triggered:

1. **Real-time webhook** — X calls `POST /api/webhook/x` when someone mentions `@trypitchdotco`. The server hands the mention to a fresh `opencode run` session running the `x-mention` skill and returns `200` immediately.
2. **Manual / cron** — `pitch-cli server` also exposes `/api/webhook/trigger` (curl on demand), and the `discover` CLI polls X for ICP leads.

2 The CLI Surface

Binary: `./target/release/pitch-cli`.

```
pitch-cli server --port 8790              # Axum webhook dispatcher (always-on)
pitch-cli discover [--max N] [--dry]      # search X for ICP SaaS prospects → CRM
pitch-cli budget                          # daily caps + rolling 1h burst usage
pitch-cli circuit-breaker [--trip "reason" | --reset]   # safety pause / resume
pitch-cli sync                            # DB summary (jobs, delivered, prospects)
pitch-cli db jobs [--status X]            # list mention jobs
pitch-cli db get-job <tweet_id>           # job by tweet id
pitch-cli db upsert-job '<json>'          # upsert a mention job
pitch-cli db prospects [--stage X]        # list CRM prospects
pitch-cli db get-prospect <handle>        # prospect by handle
pitch-cli db upsert-prospect '<json>'     # save / update a prospect
pitch-cli db log '<json>'                 # log an activity
pitch-cli db insights [get | set]         # adaptive memory blob
```

Removed subcommands: `x-api`, `mcp`, `inbox`, `worker`, `trigger`. Reads go through the `xmcp` MCP server tools inside opencode; writes go through agent-webbridge. Pitch video jobs go through the `pitch` MCP server tools.

3 Webhook Endpoints → Skill Mapping → Business Accomplishments

All webhook routes are exposed under the single unified base **`/api/webhook`** (`src/server.rs`). Every incoming event dispatches a fresh `opencode run` session that executes the mapped skill.

Endpoint	Triggered Skill	Pipeline Execution	Business Accomplishment (ROI)
`POST /api/webhook/x`	`x-mention`	1. Dispatch opencode session with the mention 2. Receipt ack reply via webbridge `Testing` 3. Pitch MCP render (1080p MP4) 4. In-thread S3 video delivery	**Inbound Viral Demo Funnel:** users receive a studio-quality video demo of their URL in minutes; public replies showcase PITCH's magic to all followers, driving viral social proof & signups.
`POST /api/webhook/pitch`	`x-mention`	1. Dispatch opencode session on completion 2. Session polls `get_job`, posts S3 delivery reply	**Fast Delivery:** a render completion can prompt an immediate delivery pass instead of waiting for the next poll.
`POST /api/webhook/trigger` `{"action":"growth"}`	`x-growth`	1. Dispatch opencode session running the x-growth session loop 2. Prospect/search, engage, outreach as appropriate 3. Upsert CRM (`state/prospects.jsonl` + SQLite)	**High-Intent Lead Pipeline:** builds a pipeline of SaaS founders who need video walkthroughs.
`POST /api/webhook/trigger` `{"action":"mentions"}`	`x-mention`	1. Dispatch opencode session to check recent mentions 2. Handle any unprocessed `@trypitchdotco` mentions	**Inbound Recovery Net:** recovers mentions even if an X webhook callback fails or drops.
Scheduled Agent Pass `(opencode run --agent x-growth)`	`x-engage` `x-outreach`	1. Like/reply to warm leads (`stage: warming`) 2. Custom DM with user's URL walkthrough 3. Update CRM (`stage: contacted / in_convo`)	**Paid Subscriber Conversion:** nurtures SaaS founders in public, then converts them in DMs into paid PITCH subscribers.
Daily Scheduled Pass `(opencode run --agent x-growth)`	`x-content` `x-community`	1. Post product updates & tech takes 2. Help indie hackers in public threads 3. Drive organic profile impressions	**Brand Authority & Awareness:** builds trust, follower growth, and organic inbound traffic for `@trypitchdotco`.
`GET /api/webhook/x`	System Utility	HMAC-SHA256 CRC token verification challenge	**X API Compliance:** required by X Account Activity API to register webhooks.
`GET /api/webhook/health`	System Utility	Liveness probe returning HTTP 200 OK	**Deployment Health:** ensures the server is operational.
`GET /api/webhook/stats`	System Utility	Pipeline metrics query from SQLite	**Observability:** monitors total jobs, renders, and CRM prospects.

4 Request Flows

4.1 Flow A — Real-time mention → demo delivery (x-mention skill)

Endpoints: `GET /api/webhook/x` (CRC handshake) and `POST /api/webhook/x` (event callback).

CRC registration (GET). `GET /api/webhook/x?crc_token=<token>` returns `{"response_token": "sha256=<base64(HMAC-SHA256(crc_token, X_CLIENT_SECRET))>"}` (`handle_crc` in `src/server.rs`).

Event callback (POST). The handler parses `tweet_create_events` for a mention of `@trypitchdotco` from another account, builds a task prompt, and returns `200 {"status":"ok","dispatched":true}` immediately.

Session dispatch (`spawn_opencode_session` , `src/server.rs`): spawns `opencode run "<task>"` in the repo root (so it picks up `opencode.jsonc` , the skills, and the `x-growth` agent), caps concurrency via `MAX_OPENCODE_SESSIONS` (default 3), and writes the session's stdout to `data/sessions/<timestamp>.log` .

Inside the session, the `x-mention` skill does:

1. `awb status` first; then posts an instant receipt reply via `agent-webbridge` (`Testing` profile).
2. Calls the Pitch MCP tool `create_demo_video` for the product URL in the tweet. No URL or an `s3.trypitch.co` URL → records the job as `no_url_found` .
3. Polls `get_job` until `COMPLETED` (5 min initial sleep, then 2-min intervals), extracts the S3 artifact URL, and delivers it as an in-thread reply via `agent-webbridge` . FAILED → `failed` , otherwise retries.
4. Records each step in SQLite (`mention_jobs`) and the CRM state files.

Render completion (POST `/api/webhook/pitch`). If `PITCH_WEBHOOK_URL` is set on `create_demo_video` , Pitch MCP posts the completion callback here. The handler dispatches another opencode session to look up the job and deliver the finished video. Sessions also poll on their own, so this callback is optional (belt-and-suspenders).

4.2 Flow B — Manual trigger endpoint

`POST /api/webhook/trigger` with optional JSON body `{"action": "..."}` (`src/server.rs`). Dispatches an opencode session:

- `action` = `growth` or `session` → runs the `x-growth` session loop.
- `action` = `discover` → runs the `x-prospect` skill.
- otherwise → runs the `x-mention` recovery pass.

Responds `200 {"status":"ok","action":...,"dispatched":true}` immediately.

4.3 Flow C — Outbound prospect discovery (x-prospect skill)

`discover_prospects(max_per_query, dry_run)` (`src/discover.rs`), triggered via `pitch-cli discover` or the webhook `action=growth` path:

1. Runs the fixed ICP search-query list (`src/discover.rs`) against X search recent (`GET /tweets/search/recent`).
2. Skips `@trypitchdotco` , `@adnanspitch` , and already-known handles.
3. Scores each lead 1–10 (`calculate_lead_score` , `src/discover.rs`) using URL presence, competitor keywords (tella, screen studio, loom, guidde, supademo, tango), and intent keywords (need, looking for,

alternative, how to, launched, building).

4. Builds a pre-cooked DM hook (`generate_pitch_hook`) and upserts the prospect into `prospects` (stage `new`, segment `founder`) plus `state/prospects.jsonl`.

4.4 Flow D — Health, stats & observability

- `GET /api/webhook/health` → `{"status":"ok", ...}` (`src/server.rs`).
- `GET /api/webhook/stats` → counts of jobs (`mention_jobs_total`, `delivered`, `rendering`) and `prospects_total` (`src/server.rs`).
- `pitch-cli sync` → same summary to stdout.
- `pitch-cli budget` → remaining daily caps per action + rolling 1-hour burst count (`src/safety.rs`). Writes are blocked when a cap is 0 or the breaker is tripped.
- `pitch-cli circuit-breaker` → `OK to run` / `PAUSED`; trips are recorded in `state/circuit-breaker.jsonl` and 3 trips in 24h create `state/HARD_STOP` (hard pause) until `--reset`.

5 Webhook Routes Table

Server binds `0.0.0.0:{PORT}`, where `PORT` defaults to `8790` (`src/server.rs`). All webhook routes live under the single canonical base `/api/webhook` — no duplicated mounts or aliases. X registers `https://<public-url>/api/webhook/x` as its one callback URL; Pitch MCP completion posts to `https://<public-url>/api/webhook/pitch` (`PITCH_WEBHOOK_URL`).

Method	Path	Purpose
`GET`	`/api/webhook/x`	X webhook CRC challenge (HMAC-SHA256 token)
`POST`	`/api/webhook/x`	X mention event → dispatch `x-mention` opencode session
`POST`	`/api/webhook/pitch`	Pitch MCP render completion → dispatch delivery session
`POST`	`/api/webhook/trigger`	Manual/on-demand dispatch (`{"action":"mentions"|"growth"|"discover"}`)
`GET`	`/api/webhook/health`	Liveness probe
`GET`	`/api/webhook/stats`	DB pipeline counts

6 Job & Prospect State Machines

`mention_jobs` table (one row per tweet):

```
no_url_found → (terminal; nothing to render)
      submitted → rendering → delivered
                      |
                      └───────→ failed
```

Job rows are written by the dispatched opencode sessions as they work; the `x-mention` skill is responsible for idempotency (never double-reply or double-bill a render for the same tweet).

Prospect stages (`prospects.stage`): `new` → `warming` → `contacted` → `in_convo` → `trial` / `customer` ; terminal `do-not-contact` (permanent, on any opt-out) and `lost` . Stages advance only on real signals; a `new` prospect is never DM'd before the warm-up bar is met.

7 Database Schema & State Mirroring

`data/pitch_bot.db` (override with `SQLITE_DB_PATH`), WAL mode, tables created at startup (`src/db.rs`):

- **`mention_jobs`** — idempotent demo pipeline state (unique `tweet_id`).
- **`prospects`** — CRM; upsert keyed on `handle` , mirrored to `.opencode/skills/x-growth/state/prospects.jsonl` .
- **`activities`** — every action (`ts` , `action` , `handle` , `result`), the input for `budget` . Mirrored to `activity-log.jsonl` .
- **`insights`** — adaptive memory blob (single-row table, `id=1`).

CRM state files live in `.opencode/skills/x-growth/state/` : `account.json` (operating identity: `@adnanspitch` operator, `@trypitchdotco` product, ramp dates), `prospects.jsonl` , `activity-log.jsonl` , `insights.md` .

8 Schedules & Cron

The binary has **no internal scheduler.** Automation = the always-on webhook server, which dispatches an `opencode run` session per event. Directory used in examples: `/opt/pitch-xbot` (repo root).

8.1 The recovery net (why you still need cron)

The webhook is the primary path. If an event is dropped, hit the trigger endpoint on demand — e.g. a cron line:

```
# every 5 min: re-check recent mentions + deliver any completed renders
*/5 * * * * curl -s -X POST http://127.0.0.1:8790/api/webhook/trigger -H 'Content-Type: applicat
```

8.2 Server (always-on)

```
./target/release/pitch-cli server          # binds 0.0.0.0:8790
# or: PORT=9000 ./target/release/pitch-cli server
```

Run under a process manager (launchd on macOS, systemd on Linux; or nohup, tmux, pm2). It must be reachable by X on a public URL (e.g. ngrok tunnel or reverse proxy) for webhooks to arrive. Each event spawns `opencode run` (concurrency capped by `MAX_OPENCODE_SESSIONS` , logs under `data/sessions/`), so the host also needs the opencode CLI and the model provider authed.

8.3 Outbound prospecting (cron)

ICP discovery is rate-sensitive — spread it out, never burst. `discover` is blocked while `budget` shows 0 remaining for the `discover` action (cap 40/day default, scaled by ramp factor during cold start — `src/safety.rs`).

```
# every 3 hours, 08:00-23:00 ASIA/KOLKATA
0 8-23/3 * * * cd /opt/pitch-xbot && ./target/release/pitch-cli discover --max 5 >> /var/log/pit
```

8.4 Schedule summary

Pass	Cadence	Rationale
`server`	always-on	real-time mention → demo via dispatched sessions
`/api/webhook/trigger` `{"action":"mentions"}`	every 5 min	recovery net; re-checks mentions + delivers renders
`discover`	every 3 h	ICP lead discovery, spaced for rate limits
`circuit-breaker` + `budget` + `sync`	daily 09:00	health + budget heartbeat

X burst hygiene: ≥90s between consecutive writes; the safety engine enforces a 10-actions/hour burst signal via `budget`.

9 Safety Engine

9.1 Circuit breaker

- `pitch-cli circuit-breaker` → `OK to run (N trips in 24h)` or `PAUSED ...` (exit 1). If PAUSED, automation must stop.
- `pitch-cli circuit-breaker --trip "<reason>"` records a trip in `state/circuit-breaker.jsonl`; **3 trips in 24h** create `state/HARD_STOP`, which pauses automation until a human runs `--reset`.
- Kill-switch rule: on any CAPTCHA / warning / limit / repeated failure, trip the breaker and stop.

9.2 Budget caps (defaults, scaled ×0.25 during cold-start ramp)

Until `state/account.json` `ramp_until` (2026-08-30), caps are multiplied by 25% (never below 1). `pitch-cli budget` reports caps, used, remaining, actions in the last hour, and a burst warning at ≥10 actions/hour.

Action	Daily cap (full)	Daily cap (cold-start ×0.25)
like	50	12
reply	15	3
follow	15	3
dm	10	2
post	4	1
quote	4	1
discover	40	10

Session rules: max 3 sessions/day, min 2h apart; no outbound DMs before 2026-08-23; max 2 DMs/hour after the gate lifts; no two outbound messages identical.

10 X API v2 & Pitch MCP

10.1 X API v2 (`xmcp` MCP server)

Reads go through the official X MCP server (`https://api.x.com/mcp`) via the local `xurl` bridge (`opencode.jsonc`), authenticated with OAuth2 PKCE using `X_CLIENT_ID` / `X_CLIENT_SECRET` (first run opens the browser, tokens cached in `~/.xurl`). Tools: `me`, `lookup_user`, `search`, `mentions`, and more. The old internal X API client (`src/x_api.rs`) is legacy and **blocked**: its OAuth2 user tokens in `.env` are expired. The single remaining internal X call is `pitch-cli discover`.

10.2 X writes (agent-webbridge)

All X writes (post, reply, like, DM) go through agent-webbridge driving real Chrome (`Testing` profile, router `127.0.0.1:10086`). `awb status` must show `extensionConnected: true` before any write. Never use a headless X API client.

10.3 Pitch MCP (`pitch` MCP server)

`https://api.trypitch.co/mcp`, Bearer `PITCH_API_KEY`. Tools: `create_demo_video` (costs 3 credits), `get_job`, `get_credits`. These are called from inside opencode sessions (`pitch` MCP server configured in `opencode.jsonc` with `{env:PITCH_API_KEY}`).

11 Setup & Environment

11.1 Requirements

- Rust 1.80+ (`cargo` / `rustc`)
- opencode CLI with a model provider authed
- agent-webbridge installed (`npm i -g agent-webbridge`), fleet up for `Testing` profile
- A reachable public endpoint for the webhook server (or tunnel)

11.2 `.env` (gitignored — see `src/config.rs`)

```
X_CLIENT_ID=your_x_client_id
X_CLIENT_SECRET=your_x_client_secret
X_OPERATOR_HANDLE=@trypitchdotco
X_USERNAME=your_x_username
X_PASSWORD=your_x_password
PITCH_API_KEY=your_pitch_api_key
PITCH_WEBHOOK_URL=https://<public-url>/api/webhook/pitch   # optional
SQLITE_DB_PATH=./data/pitch_bot.db      # optional override
PORT=8790                               # optional override
MAX_OPENCODE_SESSIONS=3                 # optional: concurrent dispatched sessions
X_WEBHOOK_ID=                           # optional: registered webhook id
```

- `PITCH_API_KEY` is consumed by the `pitch` MCP server in `opencode.jsonc` (`{env:PITCH_API_KEY}`); opencode resolves it from the process env, so export it before launching opencode (`set -a; source .env; set +a`).
- `X_CLIENT_ID` / `X_CLIENT_SECRET` feed the `xmcp` `xurl` bridge OAuth2 login.
- Legacy keys (`X_API_KEY`, `X_API_SECRET`, `X_BEARER_TOKEN`, `X_USER_*`) are unused by the code.
- Never commit `.env` or expose `PITCH_API_KEY` / OAuth2 tokens in logs.

11.3 Build & verify

```
cargo build --release
./target/release/pitch-cli sync                     # empty DB → zeros
./target/release/pitch-cli circuit-breaker          # expect "OK to run"
./target/release/pitch-cli budget                   # confirm caps + 0 used
./target/release/pitch-cli server --port 8790 & # boot server
curl http://localhost:8790/api/webhook/health   # ok
curl "http://localhost:8790/api/webhook/x?crc_token=test"  # sha256=...
curl -X POST http://localhost:8790/api/webhook/pitch -d '{"jobId":"x","status":"COMPLETED"}'
        # ok
```

Verify the MCP servers load in opencode: `opencode debug config` and `opencode mcp` list `pitch` and `xmcp`. The `pitch` server needs `PITCH_API_KEY` exported; `xmcp` needs `X_CLIENT_ID` / `X_CLIENT_SECRET` and a one-time browser login on first use (or `xurl auth oauth2`).

11.4 Wire the webhook in X

Register `https://<public-url>/api/webhook/x` as the Account Activity webhook URL in your X app. X calls `GET` (CRC) during registration and `POST` on every mention thereafter.

12 Automation Checklist

1. `.env` populated with `PITCH_API_KEY` and `X_CLIENT_ID` / `X_CLIENT_SECRET`; `xmcp` one-time login done; `awb up "Testing"` running.

2. `cargo build --release` passes.
3. `pitch-cli circuit-breaker` says `OK to run`; `pitch-cli budget` shows remaining caps.
4. `pitch-cli server` is always-on behind a public URL (dispatches sessions capped by `MAX_OPENCODE_SESSIONS`, logs in `data/sessions/`).
5. Optional `*/5` cron curls `/api/webhook/trigger` `{"action":"mentions"}`; `*/3`h cron runs `pitch-cli discover`.
6. Watch `data/sessions/*.log` + `GET /api/webhook/stats`; trip/reset the breaker on anomalies.